
accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000



Designing and Coding Your Application for Global Use

Ezekiel Sanborn de Asis
DTS Engineer,
Palm, Inc.

PalmSource
Conference 2000

The Sales Pitch

- Roughly 30% of Palm Powered™ devices are sold internationally
- Improvements in the OS and Tools have made (and will continue to make) the Localization process easier
- Improvements in the OS have made it easier to adapt your code to be locale independent

A Brief History of Time

Palm OS 3.1

- Japanese OS
- Text Manager
- International Manager
- Shift-JIS Character Encoding

Palm OS 3.5

- Overlay Manager
- Custom Date/Time Formats
- TxtParamString

Palm OS 4.0

- Locale Manager
- Locale Module
- New Time Zone Support
- Table Driven Text Manager

Use Palm OS Glue Functions on Older Versions of the OS! Linkable Lib that provides backwards compatibility

Text Manager (3.1 OS)

- Character Sizes, Attributes and Encodings
- Word Bounds and String Truncation
- Searching, Comparing and Transliterating Text
- Previous and Next Characters
- TxtFindString
- TxtParamString from 3.5 OS
- TxtCovertEncoding from 4.0 OS
- FntCharWidth, TxtCharWidth, FntWCharWidth
- Table Driven from 4.0 OS

Overlay Manager (3.5 OS)

- Separate Localized Resource DB
- One to One Mapping from Base Resource DB to Overlay Resource DB
- Overlay Manager automatically substitutes localized overlay resources at runtime
  - You usually do not call the overlay manager
  - If overlay matching current locale exists, the overlay manager will substitute it in
  - You can specify not to use overlays, or call the overlay manager to force the use of an overlay that does not match the default system locale

Date Time Formats

- Date Format in System Preferences
- DateTemplateToAscii from 3.5 OS
- Date Format moved out of ‘cnty’ resource in 4.0 OS (see locale manager)
- Time Zone support from 4.0 OS
  - Date Time Panel with Time Zone and DST on/off
  - Not automatic Daylight Savings Time (DST)
  - No automatic adjustment of appointments
  - Not a complete list of cities and Time Zones
- Be careful trying to get localized string values directly from system resources

Locale Module (4.0 OS)

- Resource DB specific to a character encoding defining OS behavior relative to that encoding
- Many Locales can be based on 1 Locale Module
- Latin and Shift-JIS Locale Modules
- Contains
  - Data and Tables to drive Text Manager
  - Resources and Displays (example: Keyboard)
  - Fonts
  - Code to patch out OS routines
- The end result is that it is easier for us to create other locales

Locale Manager (4.0 OS)

- Provides a consistent method to get and set locale settings
  - Language, country, Date Format, Time Zone, etc.
  - See LocaleMgr.h for complete listing
  - **NOTE: Use LmChoiceLocale over LmChoiceLanguage or Country
  - **`LmGetLocaleSetting`**
  - **`LmLocaleToIndex`**
  - **`LmGetNumLocales`**
  - **`Use LmGlueXXX for older OS versions`**
- ‘cnty’ resource removed
- OmLocaleType replaced by LmLocaleType

Globalizing Your Code

- Follow the formats set by the user for Dates, Times, numbers, etc.
- Leave room for longer foreign text
- Prepare for other character encodings
- Consider double byte characters when manipulating, comparing or traversing strings
- Use templates for text generated at run time
- Prepare to be overlayed
- Make Base Resources Locale Independent
- Take advantage of the Palm OS Glue Library

Formats

- For Preferences that might be customized by the user (country, date format, etc.)
  - Use PrefGetPreference with appropriate SystemPreferencesChoice selector from Preferences.h
- For locale dependent formats that do not change (currency symbol, country name)

```
LmLocaleType* pUserLocale=
  (LmLocaleType*)PrefGetPreference(prefLocale);
LmLocaleToIndex(pUserLocale, &localeIndex);
LmGetLocaleSetting(localeIndex,
  lmChoiceCurrencySymbol,oValue, iValueSize)
```

- Leave room for longer formats

Character Encodings

Size Matters

- Both character's width in pixels and size in bytes are dependent on encoding
- Localization can increase field width by 50% (either direction of translation)
- String buffers for fields need to be made longer as well
- Use FntWCharWidth for pixel width of WChar

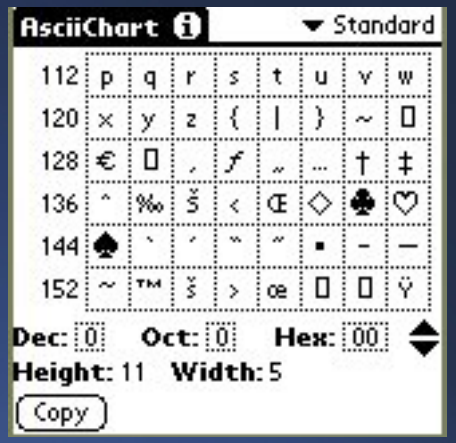


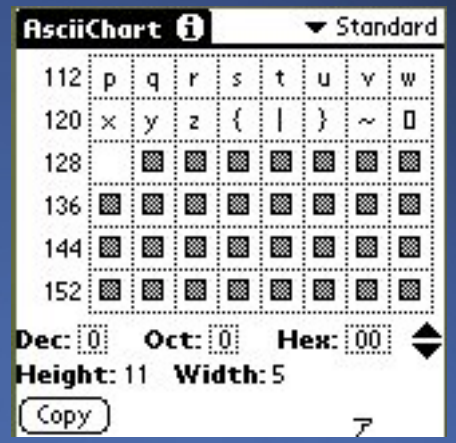


PalmSource Conference 2000

Character Encodings

- Latin, Shift-JIS, ????
- Shift-JIS defines decimal values of 129-159 and 224-239 as the first byte of a 2 byte character (2nd byte 64-126, 126-252)
- 0-127 are the same for our Latin and Shift-JIS encodings
- There are a ridiculous amount of other multi and double byte encoding schemes

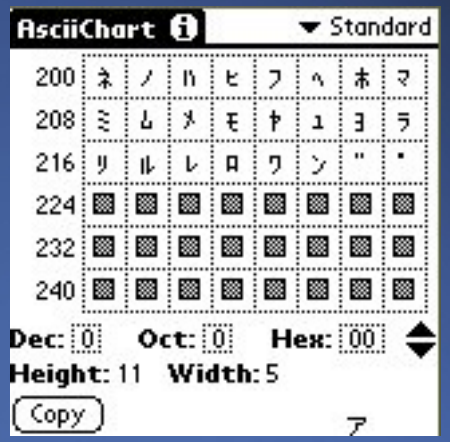




Character Encodings

- Roman letters and “katakana” exist both as single (in high ASCII) and double byte characters in Shift-JIS
- Fancy and European characters are in Latin High ASCII
- Only use High ASCII in appropriate overlays
- From 3.5 OS on Horizontal Ellipse and Numeric Space moved into control range 0x0018 and 19
  - TxtGlueGetHorizEllipsisChar
  - TxtGlueGetNumericSpace





PalmSource Conference 2000

Character Encodings

- Our virtual characters are represented with 2 bytes
- Unicode might overlap this
- In general, use the Text Manager to not rely on any specific encoding…
- Use TxtGlueCharIsVirtual to check if you have a virtual character

2 Byte Characters

Golden Rule: A Byte Is Not a Character

- Always use WChar (strings are still Char*)
- Avoid looking at 1 byte of a multi-byte character
- Make sure an offset into a string never points to the 2nd byte of a double byte character



227 can be 1st or 2nd byte in shift-jis

125 can be 2nd but not 1st in shift-jis

PalmSource Conference 2000

2 Byte Characters

- Use TxtCharIsXXX functions instead of old IsXXX macros (ex: TxtCharIsPrint not IsPrint)
- GetCharAttr will not work with double byte. Use TxtCharAttr or glue function instead
- Use TxtTransliterate for upper case and lower case transliteration

2 Byte Characters

- Use TxtGetNextChar and PrevChar to move through a string

```
width = 0;
offset = 0;
While(str[offset]){
 WChar theWChar;
 offset += TxtGetNextChar(str, offset,
   &theWChar);
 width += FntWCharWidth(theWChar);
}//for pixel width of string
```



2 4 5

2 Byte Characters

- Use TxtGetTruncationOffset to limit buffer lengths without splitting multi-byte char at end

```
buffer[TxtGetTruncationOffset(buffer,
  kMaxLen)] = 0;
```

- Use WinDrawTruncChars to do visible truncation without splitting multi-byte char at end



2 Byte Characters



- Length of found text may not be the same as the length of search string, so use TxtFindString and stash length in custom parm

```
If (TxtFindString(stringToFind,
 params->strToFind, &pos, &matchLength)){
done = FindSaveMatch(findParams,
  recordNum, pos, fieldNum,
  MatchLen,cardNum, dbID); //custom param
```

...then, in response to sysAppLaunchCmdGoTo

```
event.data.frmGoto.matchlen
 = gParam->matchCustom;
```

Flexible Base Resources

- Use TxtReplaceStr or TxtParamString to build strings at runtime
- Interface data belongs in resources
- Do not use custom resource types
- Keep numeric constants being used for sizes, widths etc. in your resources

```
UInt32 ResLoadConstant(UInt16 rscID);
```

Localization: An Overview

- Name_InCN (example Name_enUS)
  - Use PRC2OVL -showlocales for list
- Type is ‘ovly’
- ‘ovly’ =1000 that matches Base DB
- Same Creator as Base DB
- Matching Resource IDs and Checksum





PalmSource Conference 2000

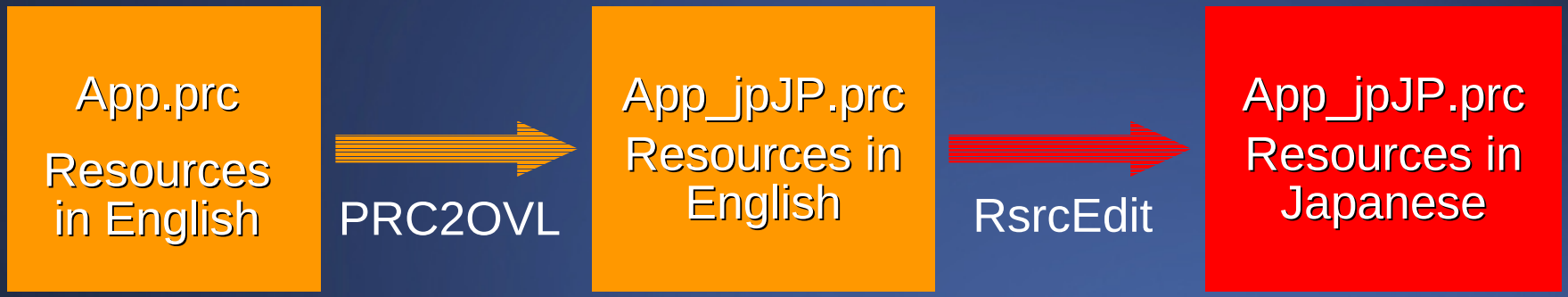
Localization: An Overview

Method 1

- Start with Base PRC (any language)
- Create Overlay with PRC2OVL Tool
- Edit resources in overlay with OnBoard RsrcEdit, PAR, PRC Constructor etc.
- This will insure that all points in previous slide are good
- You have to recreate overlay if you alter non code resources in base PRC

Localization: An Overview

Method 1



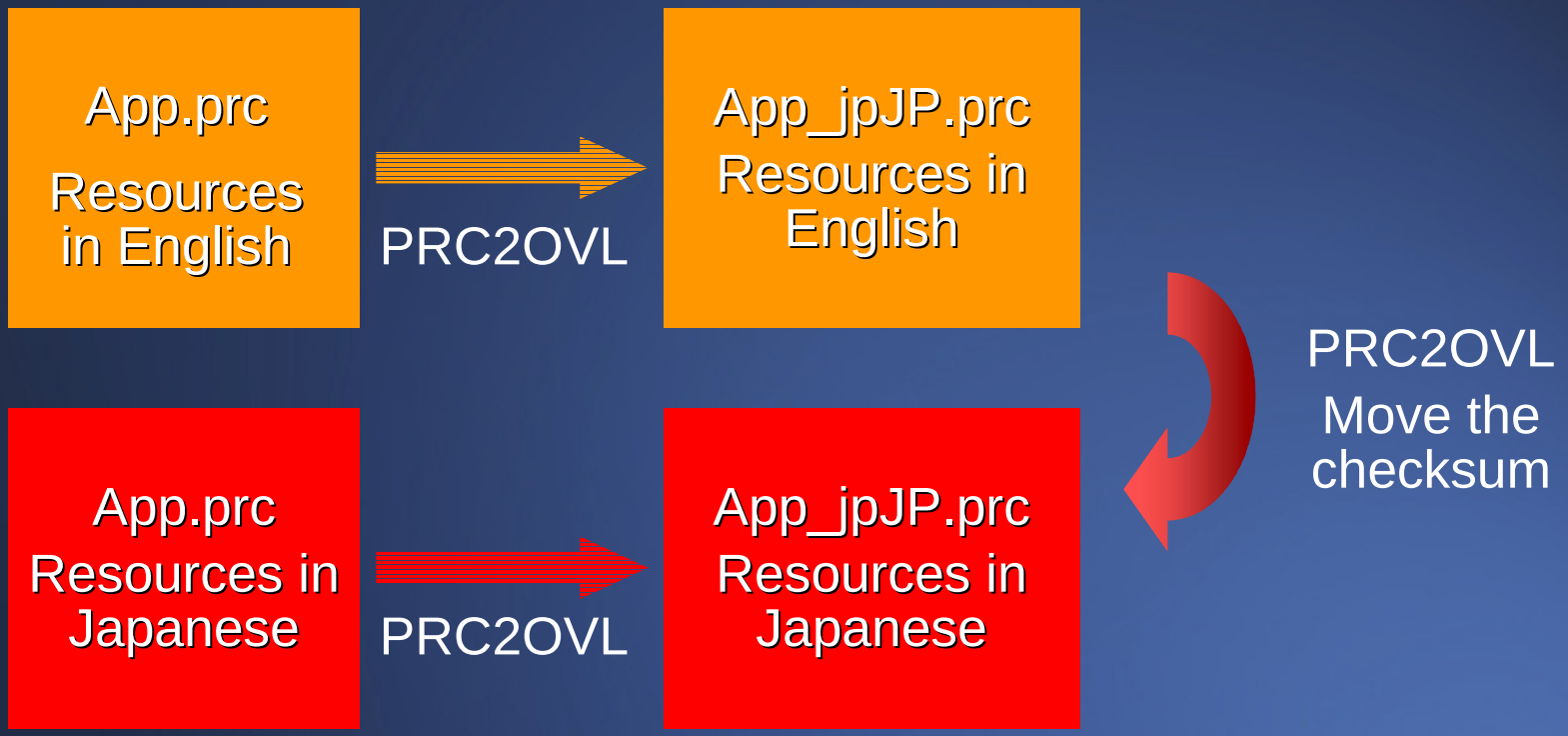
Localization: An Overview

Method 2 – not yet ready

- Create a Base Language PRC for each Language
- Choose 1 to be the shipping Base
- Create an Overlay from both PRCs
- Use the PRC2OVL tools checksum modification to make them work (specifics)
- This makes it easier to change the base PRC and only that one part of the overlay
- You have to be careful to make sure that the UI matches on both!

Localization: An Overview

Method 2 – not yet ready



Localization: An Overview

Gotchas

- Be careful about what DB you are opening
  - DmNextOpenResDatabase will return the overlay database not the app
  - DmGet1Database searches both base and overlay
  - DmGetResource searches all open databases
  - Overlay DB can only be opened in read mode

Localization: An Overview

Where to get these tools:

- PRC2OVL
  - Comes with “Other Tools” from SDK
- OnBoard RsrcEdit
  - http://www.individeo.net/
- PRC Constructor
  - Will be in the new SDK :)
- PAR
  - http://www.djw.org/product/palm/par/index.html


accelerate
collaborate
innovate
communicate
palm
PalmSource
Conference 2000